Universidad Nacional Autónoma de México

Computer Engineering

Compilers

LEXER

Students:

- 321189536
- 321184191
- 117002933
- 321134567
- 321335630

Group:
5
Semester:
2026-II

Mexico, CDMX. March 15th 2026

Contents

1 Introduction **2**

1.1 Problem Statement 2

1.2 Motivation 2

1.3 Objectives 2

2 Theoretical framework **2**

3 Body **4**

4 Results **13**

4.1 Graphical Interface 13

4.2 Test 1: Token Recognition and Whitespace Handling 13

4.3 Test 2: Comment Processing 14

4.4 Test 3: Unknown Token Detection 15

4.5 Test 4: Lexical Error Detection 16

5 Conclusions **17**

1 Introduction

1.1 Problem Statement

During the compilation process of a program, lexical analysis constitutes the first stage, where the source code is analyzed in order to identify the basic elements of the language known as *tokens*. These tokens include reserved words, identifiers, constants, operators, literals, and punctuation symbols.
The problem consists of developing a lexical analyzer capable of reading a file or a text string written in the C language and correctly classifying each element of the program into its corresponding token category.

1.2 Motivation

Lexical analysis is a fundamental stage in compiler design, since it transforms the source code into a structured sequence of tokens that will later be used by the syntactic analyzer. Implementing a lexical analyzer allows a deeper understanding of the internal functioning of compilers and enables the application of theoretical concepts such as regular expressions and finite automata.

1.3 Objectives

The main objective of this project is to develop a lexical analyzer capable of recognizing C language tokens, providing their total count and classification using regular expressions and finite automata.
The specific objectives of the project are:

- Implement a lexer that correctly identifies the different types of tokens present in the C language.
- Correctly classify tokens into the categories: keywords, identifiers, operators, constants, and punctuation symbols.
- Allow code input both from a file and from the terminal.
- Display the sequence of tokens found and their total count.

2 Theoretical framework

Lexical Analyzer: A lexical analyzer is a program that converts the input text into tokens and classifies these tokens into predefined categories. The lexical analyzer plays a crucial role in the analysis phase, since it provides the tokens to the parser to facilitate syntax analysis. In this project, the lexical analyzer reads a string or file containing C code and identifies the tokens that compose the program.

Token: A token is a sequence of characters grouped into a single entity. Each token represents a set of character sequences that convey a specific meaning within the programming language. Tokens are the basic units that allow the compiler to understand the structure of the source code.

Token Classification: In this project, the lexical analyzer classifies tokens into the following categories:

- Keywords
- Identifiers
- Constants
- Literals
- Operators
- Punctuation
- Unknown tokens

Keywords correspond to reserved words of the C language. Identifiers represent the names assigned to variables, functions, or other program elements. Constants represent numeric or character values that do not change during program execution. Literals represent sequences of characters enclosed in quotation marks. Operators represent arithmetic, logical, relational, or assignment operations. Punctuation symbols are characters used to structure the syntax of the program. Unknown tokens represent sequences of characters that cannot be classified into any valid token category.

Lexical Errors: A lexical error occurs when the lexical analyzer encounters a sequence of characters that cannot be classified as a valid token according to the defined rules. These errors may appear due to illegal characters, malformed constants, or invalid sequences that do not correspond to any token pattern. In the implementation developed in this project, these cases are classified as *Unknown tokens*, allowing the analyzer to report elements that do not belong to the lexical structure of the language.

Regular Expressions: Regular expressions are a formal notation used to describe patterns within strings. In lexical analysis, regular expressions define the structure that valid tokens must follow. Each token category can be represented by a regular expression that specifies the allowed combination of characters. For example, identifiers in the C language must begin with a letter or underscore and may be followed by letters, digits, or underscores. This rule can be described with the following regular expression:

```
[a-zA-Z_][a-zA-Z0-9_]*
```

Regular expressions are also used to describe numeric constants, string literals, operators, and punctuation symbols. These expressions allow the lexical analyzer to detect patterns in the input text and classify the tokens accordingly.

Finite Automata: A finite automaton is an abstract machine used to recognize patterns by processing symbols one at a time and transitioning between a finite number of states. The automaton determines whether a sequence of symbols belongs to a language depending on the final state reached after processing the input.

Finite Automata in Token Recognition: Since the lexical structure of most programming languages can be described using regular languages, lexical analyzers frequently rely on finite automata to recognize valid tokens. This is achieved by defining regular expressions that describe the structure of each token and then transforming those expressions into finite automata.

3 Body

The development of the lexical analyzer was carried out in several stages. First, finite automata were designed to represent the recognition of each token category of the C programming language. Then, regular expressions were derived from these automata in order to formally describe the lexical rules used by the analyzer. Finally, the lexical analyzer was implemented and a graphical interface was developed to facilitate its use.

Finite Automata Design

As the first step, finite automata were created to represent the recognition process of each token category. These automata were designed using the JFLAP tool. This allowed the visualization of states and transitions involved in the recognition of valid tokens.

Using automata helped to understand how sequences of characters are processed step by step until a valid token is recognized. Each automaton was tested with different input strings to ensure that valid tokens were accepted and invalid sequences were rejected.

Keywords Automaton

The keywords automaton recognizes reserved words of the C language. Since keywords are predefined words, the automaton processes the characters sequentially until a valid reserved word is matched.

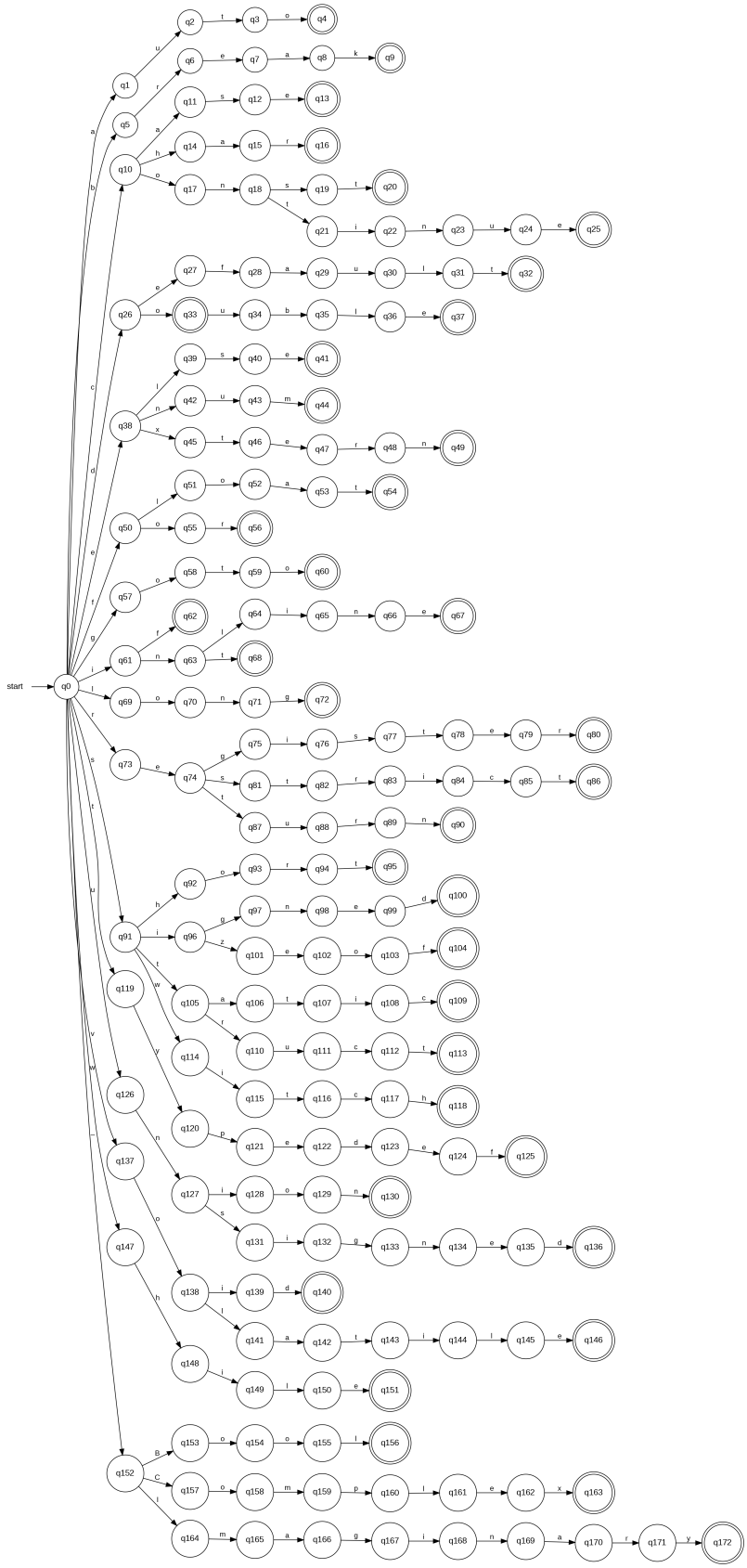

Figure 1: Automaton used to recognize keywords

Identifiers Automaton

The identifiers automaton recognizes names given to variables, functions, and other elements defined by the programmer. According to the lexical rules of C, identifiers must start with a letter or underscore, followed by letters, digits, or underscores.

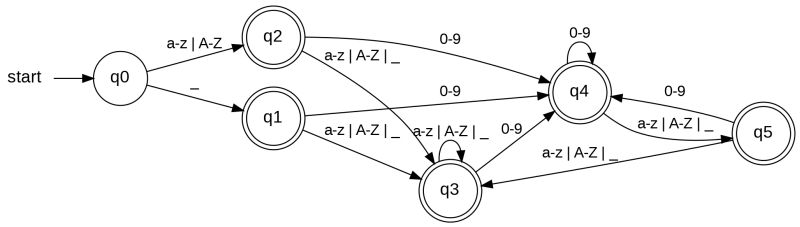

Figure 2: Automaton used to recognize identifiers

Operators Automaton

Operators represent arithmetic, logical, relational, and assignment operations. Some operators are composed of a single symbol while others are composed of multiple characters. The automaton transitions between states depending on the sequence of characters read.

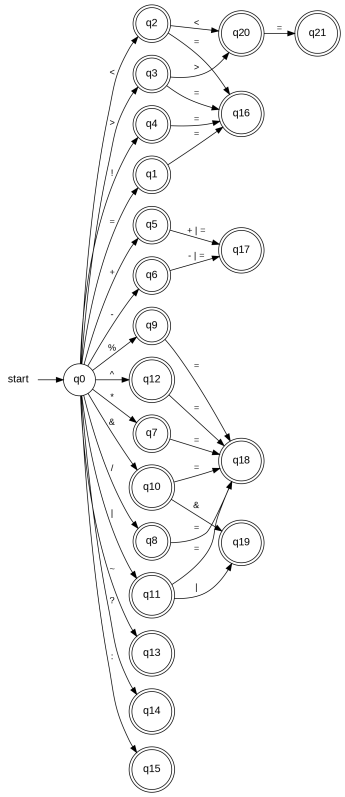

Figure 3: Automaton used to recognize operators

Punctuation Automaton

Punctuation symbols define the structure of programs in C. These include parentheses, braces, semicolons, commas, and other structural symbols.

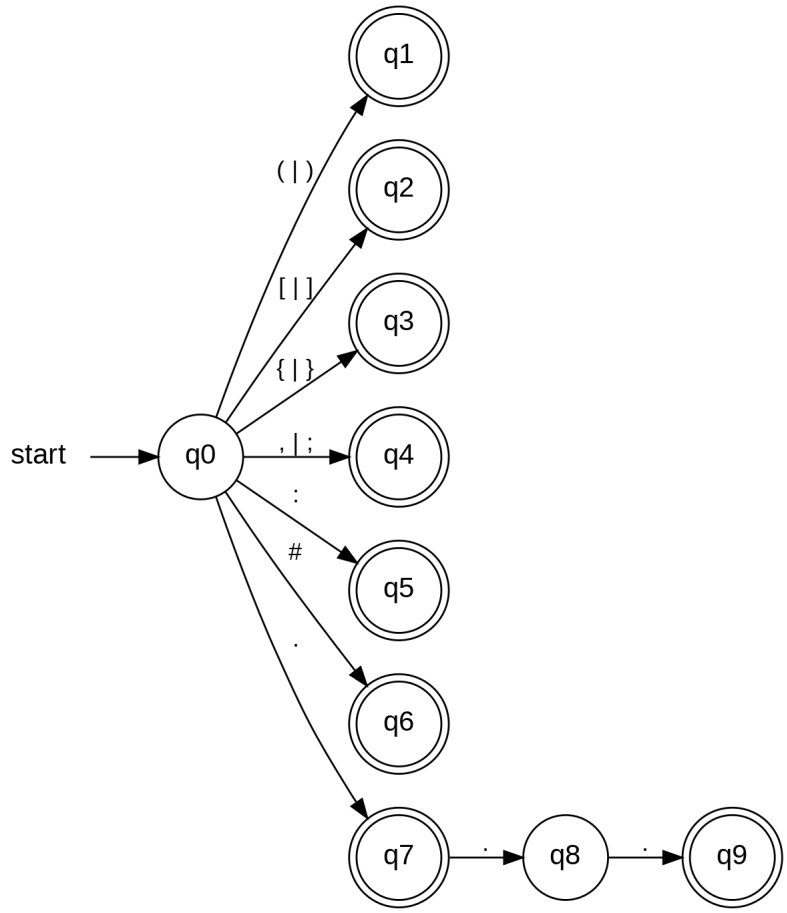

Figure 4: Automaton used to recognize punctuation symbols

Constants Automaton
The constants automaton recognizes numeric values used in the program. These include integers and floating point numbers.

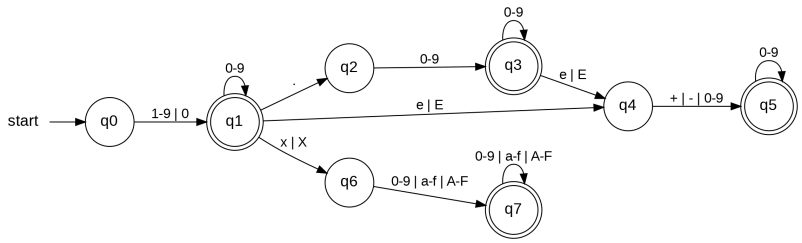

Figure 5: Automaton used to recognize numeric constants

Literals Automaton
The literals automaton recognizes string values enclosed in quotation marks.

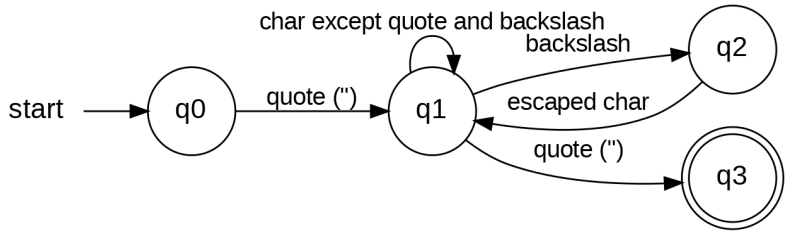

Figure 6: Automaton used to recognize string literals

Regular Expressions
After defining the automata, regular expressions were derived from them. These expressions describe the lexical structure of each token category and were used later in the implementation of the lexical analyzer.
Identifiers Regex

```
[a-zA-Z_][a-zA-Z0-9_]*
```

This expression indicates that an identifier must start with a letter or underscore followed by any number of letters, digits, or underscores.

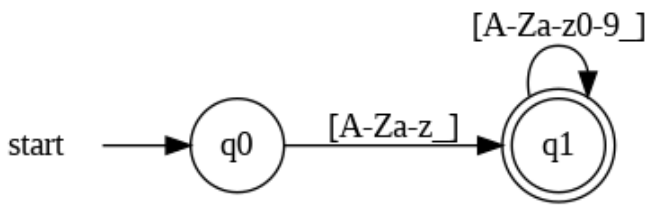

Figure 7: Regular expression for identifiers

Operators Regex

```
>>=|<<=|\+=|-=|\*=|/=|%=|==|!=|>=|<=|&&|\|\||\+\+|--|
<<|>>|&=|\|=|\^=|=|>|<|!|~|\+|-|\*|/|%|&|\||\^|\?|:
```

This expression represents the different operators supported by the lexical analyzer.

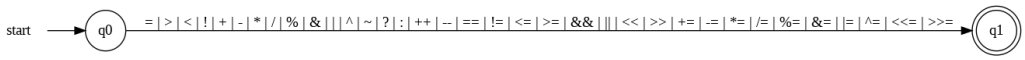

Figure 8: Regular expression for operators

Punctuation Regex

```
\.\.\.|[()\[\]{};,.:#]
```

This expression identifies punctuation symbols that structure C programs.

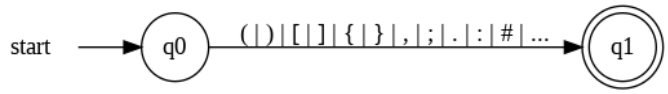

Figure 9: Regular expression for punctuation

Constants Regex

```
0[xX][0-9a-fA-F]+ | 0[0-7]+ | (0|[1-9][0-9]*) |
([0-9]+\.[0-9]+([eE][+-]?[0-9]+)?) |
(\.[0-9]+([eE][+-]?[0-9]+)?) |
([0-9]+[eE][+-]?[0-9]+)
```

This expression allows recognition of decimal, octal, hexadecimal, and floating point numbers.

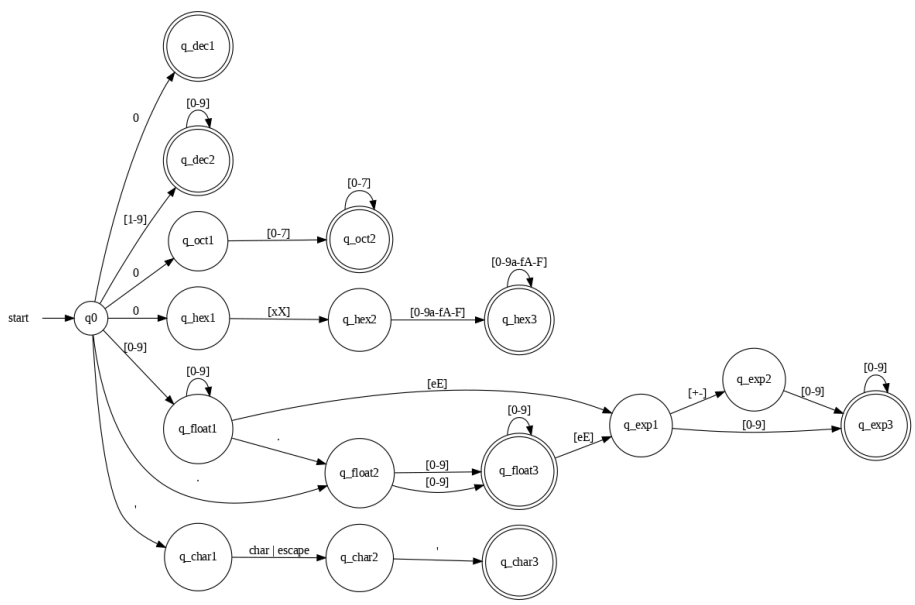

Figure 10: Regular expression for numeric constants

Literals Regex

```
"([^"\\]|\\.)*"
```

This expression recognizes strings enclosed between quotation marks.

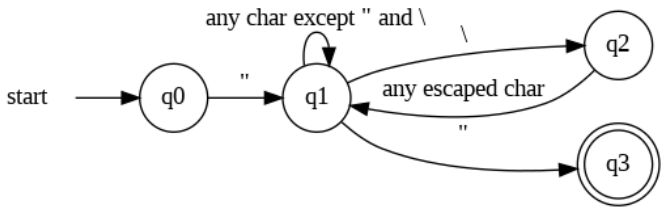

Figure 11: Regular expression for string literals

Keywords Regex

```
auto|break|case|char|const|continue|default|do|double|else|enum|
extern|float|for|goto|if|inline|int|long|register|restrict|
return|short|signed|sizeof|static|struct|switch|typedef|
union|unsigned|void|volatile|while
```

This regular expression represents the reserved words of the C language supported by the lexical analyzer. Each keyword is separated using the alternation operator (|), which allows the lexer to match any of the reserved words defined in the language specification.

Figure 12: Regular expression representation for keywords

Lexer Implementation

After defining the regular expressions, the lexical analyzer was implemented in Python. The program reads the source code either from a file or from an input string. The input text is processed line by line and comments are removed before token analysis. Each sequence of characters is compared with the defined regular expressions. If a match is found, the sequence is classified into its corresponding token category such as identifiers, keywords, constants, operators, punctuation, or literals. The tokens are stored using a hash table structure that groups tokens according to their classification. Additionally, sequences that do not match any valid token pattern are classified as *Unknown tokens*, allowing the analyzer to detect lexical errors.

Graphical Interface

A graphical interface was developed to allow users to interact easily with the lexical analyzer. The interface provides a text area where the user can enter source code or load a file. After running the analysis, the interface displays the recognized tokens grouped by category along with the total number of tokens found. This interface makes the lexical analyzer easier to test and allows users to visualize the output of the lexical analysis process.

Context Free Grammar Proposal

In order to complement the lexical analyzer, two basic CFG proposals were defined based on structures that may appear in the reading examples processed by the lexer.

Operator composed strings

Non-Terminal symbols:

```
A → Assignation expression
L → Logic expression
C → Comparison
E → Arithmetic expression
T → Term
F → Factor
ID → Identifier
NUM → Number
AO → Assignation Operator
CO → Comparison Operator
LO → Logical Operator
```

Productions

```
A → ID AO A | L

AO → = | += | -= | *= | /=

L → L LO C | C

LO → || | &&

C → E CO E | E

CO → == | != | < | > | <= | >=

E → E + T | E - T | T

T → T * F | T / F | F

F → (A) | !F | ID | NUM
```

Left Factoring
The production for comparison was originally defined as:

```
C → E CO E | E
```

Since both productions begin with the same prefix `E`, left factoring was applied:

```
C → E C'
C' → CO E | epsilon
```

This transformation removes the common prefix and improves the structure of the grammar.

Strings using If-Else structures that compare two variables
Non-Terminal symbols:

```
S → Start
IF → If-Else Structure
C → Logical Condition
B → Body
ID → Identifier
NUM → Number
CO → Comparison Operator
```

Productions

```
S → IF

IF → if ( C ) { B } else { B }

C → ID CO ID | ID CO NUM

CO → < | > | <= | >= | == | !=

B → S | ID = NUM ; | epsilon
```

Left Factoring
The condition production was originally defined as:

```
C → ID CO ID | ID CO NUM
```

Since both productions share the common prefix `ID CO`, left factoring was applied:

```
C → ID CO X
X → ID | NUM
```

This transformation simplifies the grammar and makes the condition structure more suitable for parsing.

4 Results

The following tests were conducted to evaluate the correct behavior of the lexical analyzer. These tests verify that the system correctly identifies token types, ignores elements that should not be analyzed (such as comments and whitespace), and detects lexical errors when unrecognized symbols appear in the input.

4.1 Graphical Interface

In the lexer, a small graphical interface was implemented to facilitate interaction with the analyzer. The interface allows the user to input C source code and run the lexical analysis. After execution, the program displays the recognized tokens grouped by category and the total number of tokens identified.

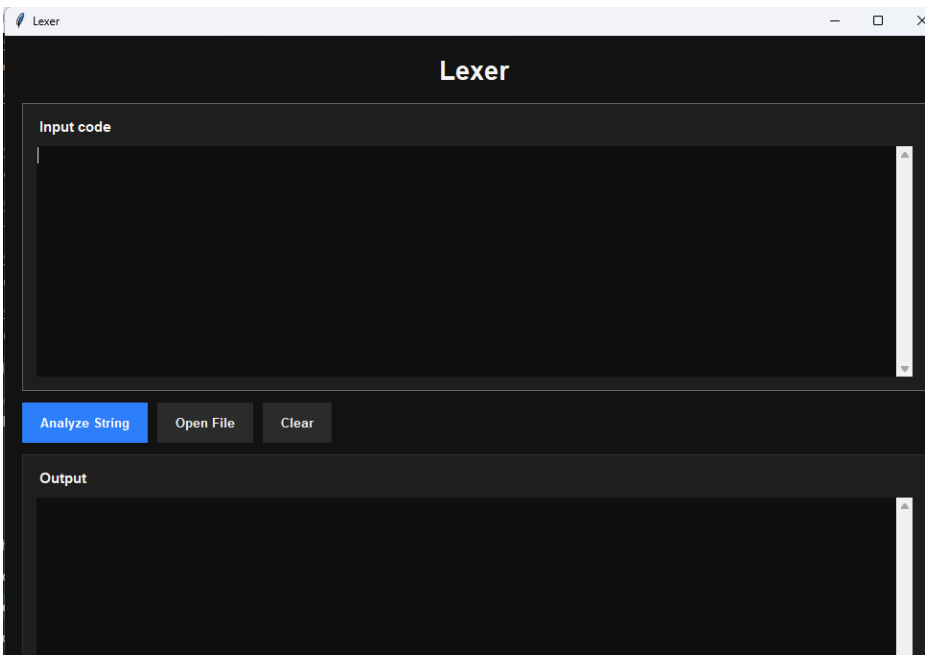

Figure 13: Graphical interface of the lexical analyzer

4.2 Test 1: Token Recognition and Whitespace Handling

In this test, a fragment of C source code was analyzed to verify that the lexical analyzer correctly identifies the different types of tokens present in a program. The system successfully recognized several token categories, including keywords, identifiers, constants, operators, functions, and literals. Additionally, the analyzer correctly reported the total number of tokens after processing the input.
Another important aspect evaluated in this test was the handling of whitespace characters. The lexer correctly handled spaces, line breaks, and tab characters. These elements were properly ignored during the analysis process, since they do not represent tokens in the lexical structure of the language.

The results confirm that the analyzer is capable of separating and classifying tokens correctly while ignoring formatting characters that do not contribute to the lexical structure of the program.

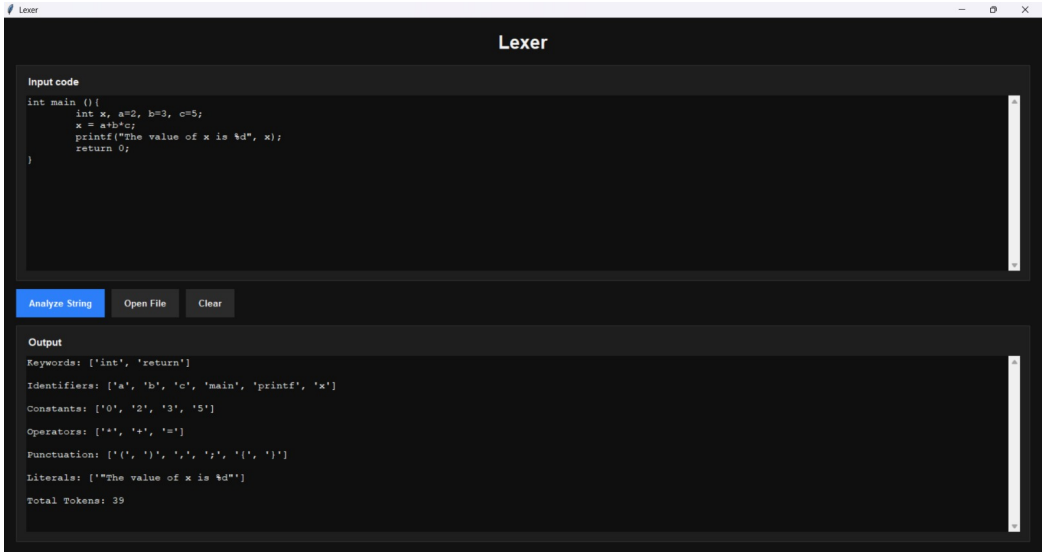

Figure 14: Test 1: Token recognition and whitespace handling in the lexical analyzer.

4.3 Test 2: Comment Processing

The purpose of the second test was to verify that the lexical analyzer correctly ignores comments present in the source code. Comments do not contribute to the syntactic structure of a program and therefore should not generate tokens during lexical analysis.

In this test, the lexer successfully removed both single-line comments (`//`) and block comments (`/* ... */`). By eliminating these elements before the tokenization process, the analyzer ensured that comments did not interfere with the recognition of valid tokens.

The results demonstrate that the lexer properly filters comments and focuses only on the meaningful elements of the program.

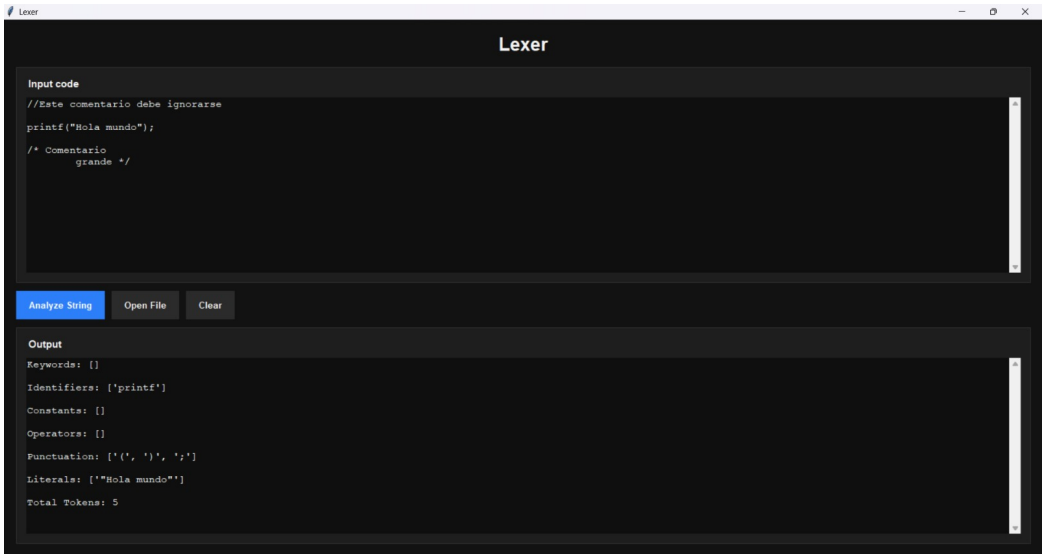

Figure 15: Test 2: Comment handling where the lexer ignores both single-line and block comments.

4.4 Test 3: Unknown Token Detection

In this test, the symbol `@` was intentionally introduced into the source code to simulate an invalid token. The objective was to verify whether the lexical analyzer could correctly detect symbols that do not belong to the defined lexical structure of the language.
During the analysis, the lexer detected this symbol as an unknown token and generated an error message indicating that an unrecognized token had been found.
This behavior confirms that the lexical analyzer is capable of detecting invalid symbols and reporting lexical errors in the analyzed source code.

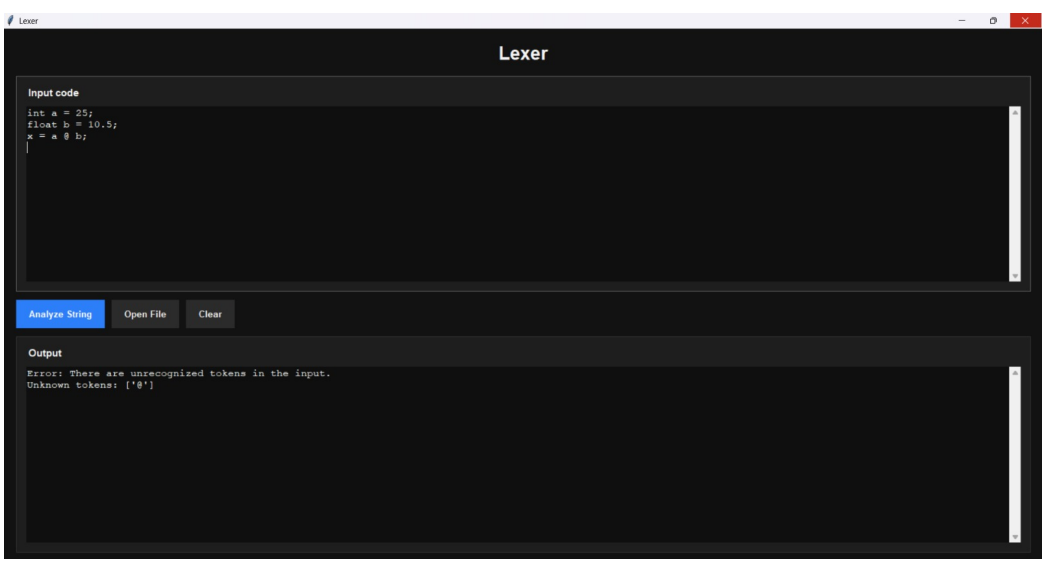

Figure 16: Test 3: Detection of an unknown token (`@`) reported as a lexical error.

4.5 Test 4: Lexical Error Detection

The objective of this test was to evaluate the ability of the lexical analyzer to detect lexical errors caused by malformed literals. Two incorrect literals were intentionally introduced into the input code:

- A string literal without a closing quotation mark.
- A character literal without a closing single quotation mark.

During the analysis, the lexer attempted to classify the tokens according to the defined lexical rules. However, since the literals were not properly delimited, they did not satisfy the patterns required for valid literals.
As a result, these fragments could not be classified into any valid token category and were reported as unknown tokens. The analyzer generated an error message indicating the presence of unrecognized tokens in the input.
This test demonstrates that the lexical analyzer correctly detects malformed literals and reports them as lexical errors, preventing incorrect code from advancing to later stages of compilation.

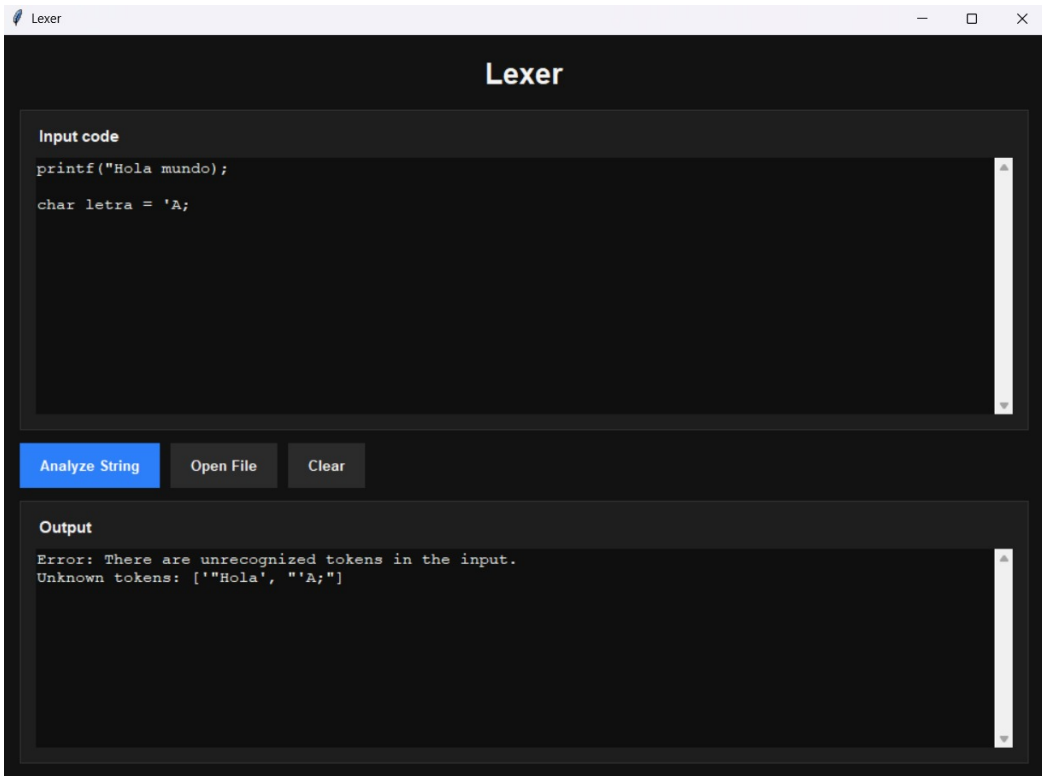

Figure 17: Test 4: Detection of lexical errors caused by malformed literals.

5 Conclusions

The development of the lexical analyzer allowed the practical application of several fundamental concepts from compiler theory. Throughout the project, it was possible to understand how programming languages formally define the structure of the tokens that compose source code and how these tokens can be identified using theoretical tools such as finite automata and regular expressions. The design of the automata made it possible to visualize the token recognition process step by step, while regular expressions provided a formal way to describe the patterns that characterize each lexical category of the C language. The implementation of the analyzer demonstrated how the lexical analysis phase simplifies source code by transforming sequences of characters into meaningful units called tokens. These tokens were classified into categories such as keywords, identifiers, constants, operators, punctuation symbols, and literals, which allowed the program information to be organized in a structured way for subsequent analysis. Likewise, the use of appropriate data structures to store and group tokens made it possible to efficiently manage the results of the analysis. Furthermore, the incorporation of a graphical interface facilitated interaction with the system, allowing users to input source code and clearly observe the tokens recognized by the analyzer. This not only improves the usability of the tool but also helps to more visually understand how lexical analysis works. Finally, the proposal of a context-free grammar illustrated how the tokens generated during lexical analysis can be combined to form valid syntactic structures within a program.

Bibliography

[1] C. Staff, "What Is Lexical Analysis?," *Coursera*, Apr. 10, 2024. [Online]. Available: `https://www.coursera.org/articles/lexical-analysis`

[2] "C tokens," *Microsoft Learn*. [Online]. Available: `https://learn.microsoft.com/es-es/cpp/c-language/c-tokens?view=msvc-170`

[3] "Introduction of Finite Automata," *GeeksforGeeks*. [Online]. Available: `https://www.geeksforgeeks.org/introduction-of-finite-automata/`